Abstract

This paper details the design, implementation, and evaluation of a mean-reversion trading algorithm grounded in stochastic calculus. The primary objective is to bridge theoretical stochastic processes with applied quantitative finance by developing a consistently profitable strategy with strict risk management protocols. This study covers the mathematical foundations of the model, system architecture, and backtesting results. Additionally, it analyzes practical implementation challenges and the engineering solutions developed to address them.

1. Introduction

1.1 Context

While traditional fundamental analysis focuses on long-term value, short-term equity market dynamics can be effectively modeled using stochastic differential equations (SDEs). This paper focuses on the Ornstein-Uhlenbeck (OU) process, which models how price shocks in the stock market revert to a historical mean. Modeling this specific SDE enables statistical inference regarding the speed and magnitude of mean reversion, as well as how inherent market noise impacts price action. The foundational research for this project involved modeling market microstructure using advanced probabilistic frameworks. This included developing Hidden Markov Models (HMMs), implementing Markov Chain Monte Carlo (MCMC) methods, and applying the Central Limit Theorem (CLT) for the statistical analysis of asset distributions.

1.2 Strategy Selection and Hypothesis

Building upon this mathematical foundation, this paper presents an algorithmic approach to mean reversion. Mean reversion is one of the most heavily documented phenomena in financial literature, resulting in a highly efficient and crowded space in modern equity markets. However, because the underlying pricing mechanisms are well-understood, it serves as an ideal framework for testing and validating applied stochastic models.

1.3 Project Objectives

Despite the competitive nature of mean-reversion strategies, the primary objective of this project is not solely to discover a novel approach to modeling mean reversion. Rather, it is to engineer a consistently profitable algorithm that prioritizes rigorous,

mathematically driven risk management. By treating the market as a continuous stochastic process, this model attempts to execute statistical arbitrage while systematically constraining downside exposure.

3. Methodology

3.1 The Ornstein-Uhlenbeck (OU) Process

To model mean-reverting behavior in equity prices, this algorithm utilizes the Ornstein-Uhlenbeck (OU) process. The dynamics of the asset price are governed by the following stochastic differential equation (SDE):

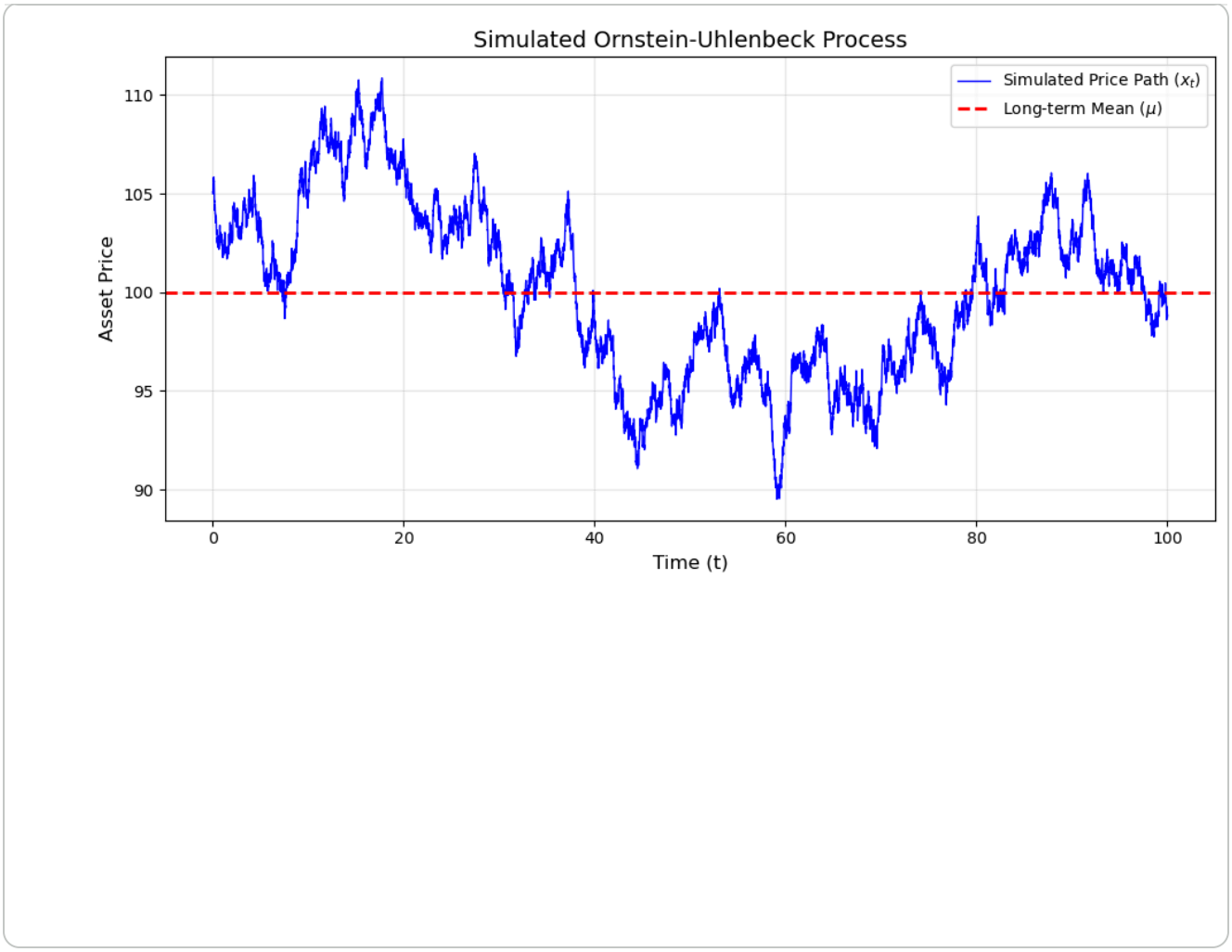
$$dx_t = \theta(\mu - x_t)dt + \sigma dW_t$$

This equation breaks the price movement down into two distinct components: a deterministic drift term that pulls the price toward an equilibrium, and a stochastic diffusion term that introduces market randomness.

The parameters of the model are defined as follows:

- x_t: The current price of the asset at time t.
- μ: The stationary (or long-term) mean. This is the equilibrium price that the asset naturally gravitates toward. Assuming the market regime remains stable, as $t \to \infty$, the expected value of the price converges to μ.
- θ: The speed of mean reversion. This dictates how aggressively the price is pulled back toward μ.
- $\theta(\mu - x_t)dt$: The deterministic drift component. The term $(\mu - x_t)$ measures the exact distance and direction of the current price from the mean. Multiplying this distance by θ models the gravitational pull bringing the price back to equilibrium over the time increment dt.
- σ: The volatility of the asset. This represents the magnitude of price impact generated by the continuous flow of buy and sell orders in the market.
- dW_t: The increment of a standard Brownian motion (or Wiener process). This captures the inherent, unpredictable random noise of the stock market.
- σdW_t: The stochastic diffusion component. It represents the overall strength of random market shocks pushing the price in unpredictable directions during the time step dt.

Show code



3.2 Parameter Estimation via Markov Chain Monte Carlo (MCMC)

The primary challenge in deploying the Ornstein-Uhlenbeck (OU) process is accurately estimating the parameters μ, θ, and σ. If the stock market were perfectly stationary, solving for these variables would be a trivial exercise. However, because equity markets continuously shift between different regimes and the intervals between volatility shocks are constantly changing, static parameter estimations inevitably break down.

While practitioners typically rely on standard Maximum Likelihood Estimation (MLE) to find point estimates, this approach fails to capture statistical uncertainty. To evaluate this uncertainty and adapt dynamically as prices mature, this model applies a Markov Chain Monte Carlo (MCMC) algorithm to generate data-driven posterior probability distributions for the parameter set $\Theta = [\theta, \mu, \sigma]$.

Data Windowing and Initialization MCMC generates probability distributions of multi-dimensional data to best explain underlying patterns by exploring high-density probability areas. The model is calibrated over a rolling window of 130 hourly ticks, representing approximately 21 days of active market data. I arrived at this specific window size through empirical and algorithmic tuning—evaluating various dataset sizes

and scoring them based on the quality of the resulting normal distributions. This testing revealed that 130 ticks provided the optimal balance, maximizing data depth without lagging behind sudden regime shifts.

To prepare the data, the 21-day set is vectorized into an array of current prices (X_t) and a shifted array representing the subsequent prices (X_{t+1}). The algorithm initializes by utilizing a baseline MLE-style estimation (derived from an AR(1) linear regression) as the prior guess to seed the starting state.

The Proposal Step From this initial guess, the algorithm iteratively generates proposed states by adding or subtracting random micro-steps drawn from finely tuned interval spaces:

$$\Theta_{proposed} = \Theta_{current} + \epsilon$$

(Where ϵ represents the tuned step-size bounds for each parameter to maintain an optimal acceptance rate).

The Exact OU Log-Likelihood Function Once defined, these proposed states are evaluated using an exact OU log-likelihood function. This function checks mathematically how well the proposed parameters describe the actual observed data transitioning from X_t to X_{t+1}. Because the transition density of the OU process is Gaussian, the algorithm calculates the sum of the log-likelihoods (LL) across the 130-tick window using the conditional expectation ($\hat{\mu}$) and conditional variance ($\hat{\sigma}^2$):

$$LL(\Theta) = \sum_{i=1}^{N} \left(-\frac{1}{2}\ln(2\pi\hat{\sigma}^2) - \frac{(X_{t+1} - \hat{\mu})^2}{2\hat{\sigma}^2} \right)$$

Where the expected mean and variance for a given time step Δt are defined as:

$$\hat{\mu} = X_t e^{-\theta \Delta t} + \mu(1 - e^{-\theta \Delta t})$$

$$\hat{\sigma}^2 = \frac{\sigma^2}{2\theta}(1 - e^{-2\theta \Delta t})$$

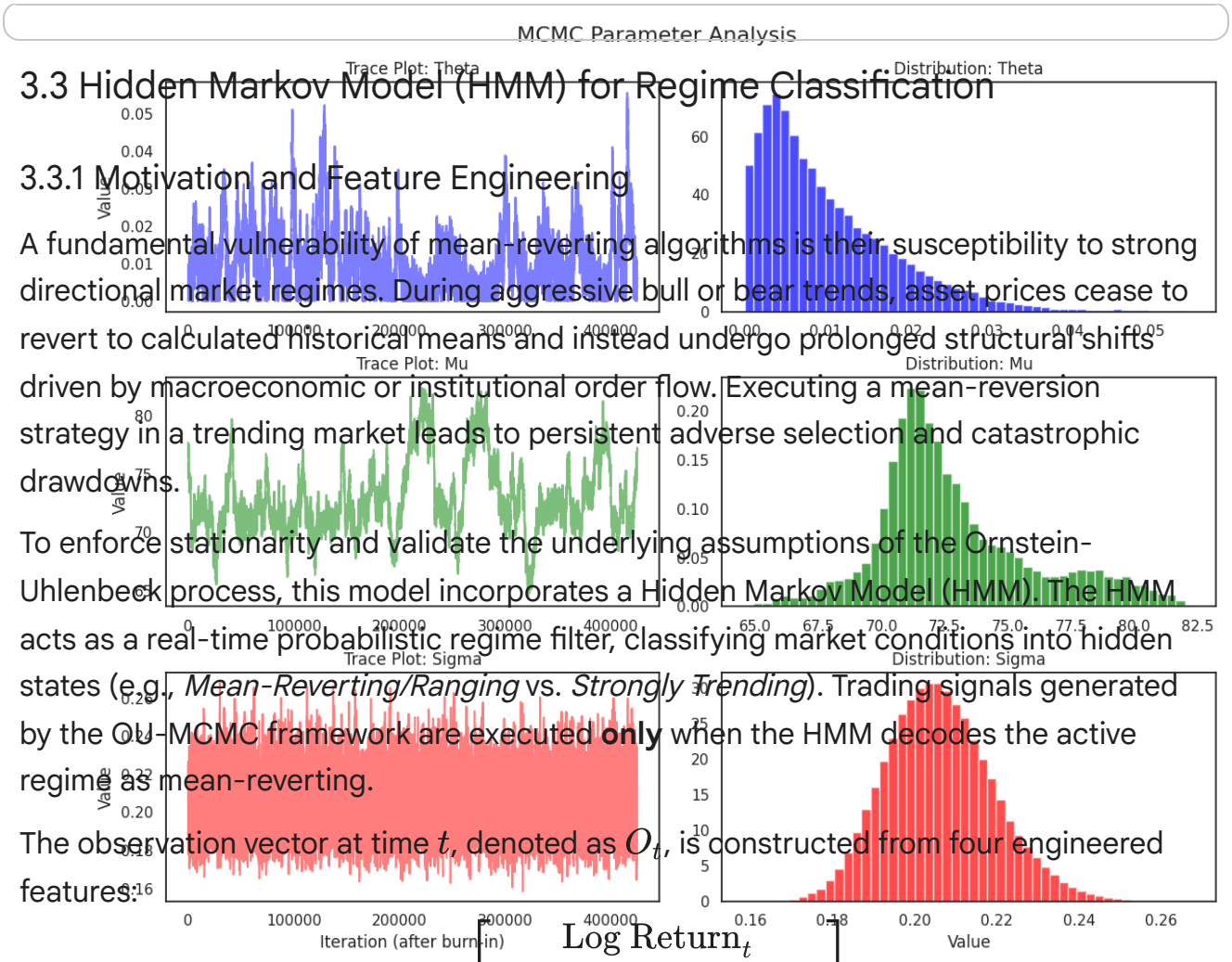
The Metropolis-Hastings Acceptance Criterion Finally, the algorithm processes these results through a Metropolis-Hastings acceptance step to determine whether to keep the proposed state. The probability of accepting the proposed parameter state (α) is calculated in log-space:

$$\alpha = \min\left(1, \exp(LL_{proposed} - LL_{current})\right)$$

If the proposed state better explains the market data ($LL_{proposed} > LL_{current}$), it is accepted deterministically. If it yields a lower likelihood, it is accepted probabilistically based on the ratio α. This stochastic acceptance prevents the chain from becoming trapped in local optima, allowing it to fully map the likely distributions of μ, θ, and σ.

After discarding a 10% burn-in period to remove initialization bias, the remaining samples form the true posterior distribution used to drive the trading logic.

Show code



3.3 Hidden Markov Model (HMM) for Regime Classification

3.3.1 Motivation and Feature Engineering

A fundamental vulnerability of mean-reverting algorithms is their susceptibility to strong directional market regimes. During aggressive bull or bear trends, asset prices cease to revert to calculated historical means and instead undergo prolonged structural shifts driven by macroeconomic or institutional order flow. Executing a mean-reversion strategy in a trending market leads to persistent adverse selection and catastrophic drawdowns.

To enforce stationarity and validate the underlying assumptions of the Ornstein-Uhlenbeck process, this model incorporates a Hidden Markov Model (HMM). The HMM acts as a real-time probabilistic regime filter, classifying market conditions into hidden states (e.g., *Mean-Reverting/Ranging* vs. *Strongly Trending*). Trading signals generated by the OU-MCMC framework are executed **only** when the HMM decodes the active regime as mean-reverting.

The observation vector at time t, denoted as O_t, is constructed from four engineered features:

$$O_t = \begin{bmatrix} \text{Log Return}_t \\ \text{Smooth Momentum}_t \\ \text{ADX}_t \\ \text{MA Spread}_t \end{bmatrix}$$

- **Log Return (r_t):** $\ln(P_t / P_{t-1})$, capturing instant price velocity.
- **Smooth Momentum:** Exponentially smoothed momentum filtering micro-structural noise.
- **Average Directional Index (ADX):** Quantifies overall trend strength, independent of direction.
- **Moving Average Spread (MA Spread):** Distance between a short-term moving average and long-term moving average ($MA_{short} - MA_{long}$), measuring trend divergence.

3.3.2 Mathematical Formulation of the HMM

The market is assumed to transition among a discrete set of K unobservable (hidden) states $S = \{s_1, s_2, \ldots, s_K\}$. An HMM is uniquely parametrized by the triplet $\lambda = (\pi, A, B)$:

1. **Initial State Probability Vector (π):**

$$\pi_i = P(q_1 = s_i), \quad 1 \le i \le K$$

where q_t denotes the hidden state at time t.

2. **State Transition Probability Matrix (A):**
$$A = (a_{ij}), \quad a_{ij} = P(q_{t+1} = s_j \mid q_t = s_i)$$
where a_{ij} represents the probability of transitioning from state s_i to state s_j.

3. **Emission Probability Distribution (B):**
$$B = b_j(O_t), \quad b_j(O_t) = P(O_t \mid q_t = s_j)$$
Because the observation vector O_t consists of continuous variables, $b_j(O_t)$ is modeled as a multivariate Gaussian distribution:
$$b_j(O_t) = \mathcal{N}(O_t \mid \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)$$
where $\boldsymbol{\mu}_j$ and $\boldsymbol{\Sigma}_j$ are the state-dependent mean vector and covariance matrix, respectively.

3.3.3 Parameter Estimation: The Baum-Welch (Forward-Backward) Algorithm

To optimize the HMM parameters $\lambda = (\pi, A, B)$ based on historical feature observations $O = (O_1, O_2, \ldots, O_T)$, the model uses the Baum-Welch algorithm—an Expectation-Maximization (EM) technique utilizing forward and backward probability variables.

1. The Forward Variable ($\alpha_t(i)$): The probability of observing the partial sequence $(O_1, O_2, \ldots, O_t)$ and being in state s_i at time t:
$$\alpha_t(i) = P(O_1, O_2, \ldots, O_t, q_t = s_i \mid \lambda)$$

- *Initialization:*
$$\alpha_1(i) = \pi_i b_i(O_1), \quad 1 \le i \le K$$
- *Induction:*
$$\alpha_{t+1}(j) = \left[\sum_{i=1}^{K} \alpha_t(i) a_{ij} \right] b_j(O_{t+1}), \quad 1 \le t \le T-1$$

2. The Backward Variable ($\beta_t(i)$): The probability of observing the remaining sequence $(O_{t+1}, O_{t+2}, \ldots, O_T)$ given state s_i at time t:
$$\beta_t(i) = P(O_{t+1}, O_{t+2}, \ldots, O_T \mid q_t = s_i, \lambda)$$

- *Initialization:*
$$\beta_T(i) = 1, \quad 1 \le i \le K$$
- *Induction:*
$$\beta_t(i) = \sum_{j=1}^{K} a_{ij} b_j(O_{t+1}) \beta_{t+1}(j), \quad t = T-1, T-2, \ldots, 1$$

Through iterative expectation and maximization steps using $\alpha_t(i)$ and $\beta_t(i)$, the algorithm updates π, A, $\boldsymbol{\mu}_j$, and $\boldsymbol{\Sigma}_j$ until likelihood convergence is reached.

3.3.4 Regime Decoding: The Viterbi Algorithm

While the Baum-Welch algorithm trains the model's parameters, it does not reveal the underlying sequence of hidden market states. To solve the **decoding problem**—finding the single most likely sequence of hidden market regimes $Q^* = (q_1^*, q_2^*, \ldots, q_T^*)$ given the observation sequence O—the model executes the **Viterbi Algorithm**.

The Viterbi algorithm uses dynamic programming to efficiently evaluate state probabilities without calculating all K^T possible state sequences.

1. Definition: Let $v_t(j)$ be the highest probability of a single state sequence accounting for the first t observations and ending in state s_j:

$$v_t(j) = \max_{q_1, q_2, \ldots, q_{t-1}} P(q_1, q_2, \ldots, q_{t-1}, q_t = s_j, O_1, O_2, \ldots, O_t \mid \lambda)$$

2. Algorithmic Steps:

- **Initialization:**

$$v_1(i) = \pi_i b_i(O_1), \quad 1 \le i \le K$$
$$\psi_1(i) = 0$$

- **Recursion (for $t = 2, 3, \ldots, T$ and $1 \le j \le K$):**

$$v_t(j) = \max_{1 \le i \le K} (v_{t-1}(i) \cdot a_{ij}) \cdot b_j(O_t)$$
$$\psi_t(j) = \arg\max_{1 \le i \le K} (v_{t-1}(i) \cdot a_{ij})$$

  (where $\psi_t(j)$ keeps track of the argument that maximized the probability, enabling backtracking).

- **Termination:**

$$P^* = \max_{1 \le i \le K} (v_T(i))$$
$$q_T^* = \arg\max_{1 \le i \le K} (v_T(i))$$

- **Backtracking (State Sequence Reconstruction):**

$$q_t^* = \psi_{t+1}(q_{t+1}^*), \quad t = T-1, T-2, \ldots, 1$$

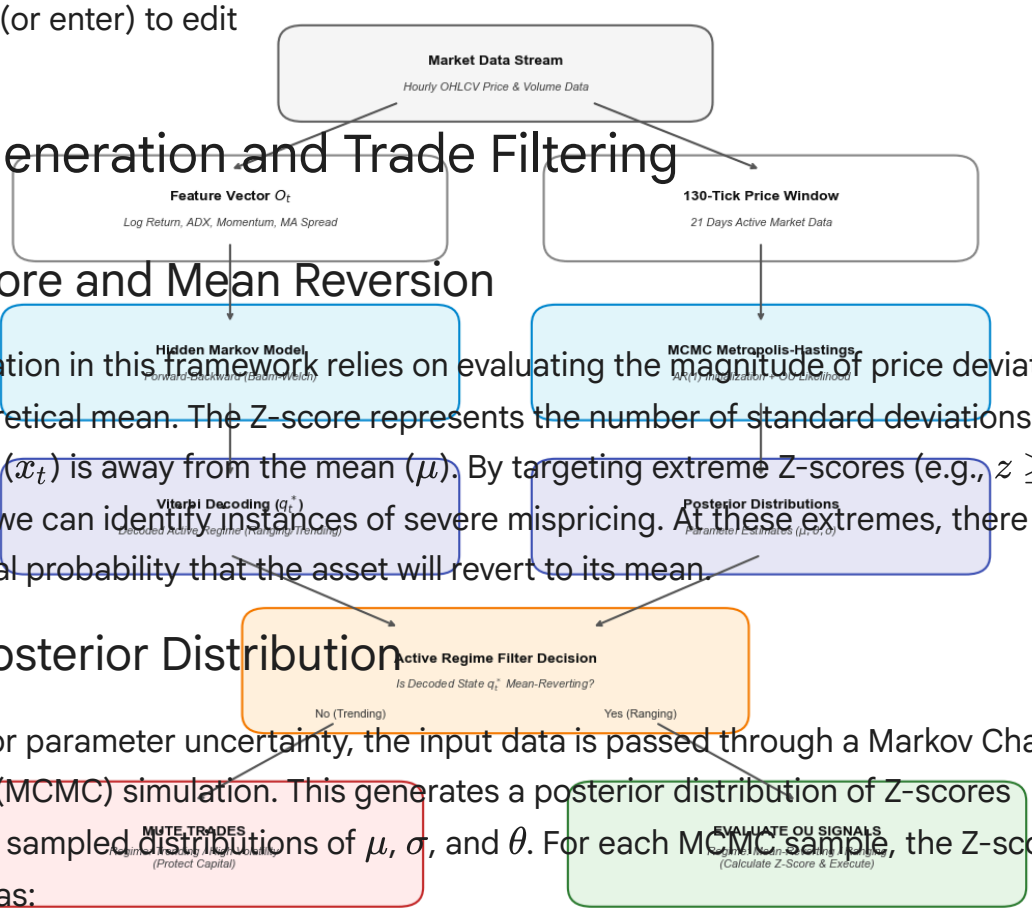
3.3.5 Integration into Overall Strategy Architecture

The Viterbi output q_t^* directly controls execution logic:

Show code

Figure 3.2: Dual-Track Integration Architecture (HMM Regime Filter + OU-MCMC Strategy)

Double-click (or enter) to edit



Signal Generation and Trade Filtering

The Z-Score and Mean Reversion

Signal generation in this framework relies on evaluating the magnitude of price deviation from its theoretical mean. The Z-score represents the number of standard deviations the current price (x_t) is away from the mean (μ). By targeting extreme Z-scores (e.g., $z \geq 2$ or $z \leq -2$), we can identify instances of severe mispricing. At these extremes, there is a high statistical probability that the asset will revert to its mean.

MCMC Posterior Distribution

To account for parameter uncertainty, the input data is passed through a Markov Chain Monte Carlo (MCMC) simulation. This generates a posterior distribution of Z-scores based on the sampled distributions of μ, σ, and θ. For each MCMC sample, the Z-score is computed as:

$$z = \frac{\mu - x_t}{\sqrt{\frac{\sigma^2}{\theta^2}}}$$

This resulting distribution models the likelihood of observing specific Z-scores, with the median serving as the most robust point estimate.

Probability Mass and Trade Filtering

Rather than relying on a single point estimate, we analyze the Cumulative Distribution Function (CDF) of our MCMC output to quantify trade viability. We calculate the percentage of the probability mass that falls beyond our target thresholds (e.g., 2 or -2). For example, if 80% of the distribution's mass exceeds the threshold, we can state with 80% confidence that an extreme mispricing event is occurring.

However, exceeding the probability threshold (set dynamically to 75%) is only the first condition for signal generation. To ensure the reliability of the distribution and limit exposure to unquantifiable risks, the signal must pass a strict filtering process before a trade is accepted:

- **Sign Conflict Check:** The system verifies that there is directional consensus across the model outputs, rejecting trades where underlying signals conflict.

- **Fat-Tail Risk (Kurtosis):** To measure and cap our exposure to outlier risks, the distribution's kurtosis must remain below a predefined limit (`max_kurtosis = 3.0`).
- **Market Regime:** The strategy restricts trading during unfavorable macroeconomic or structural market regimes (specifically blocking trades during Regime 2).

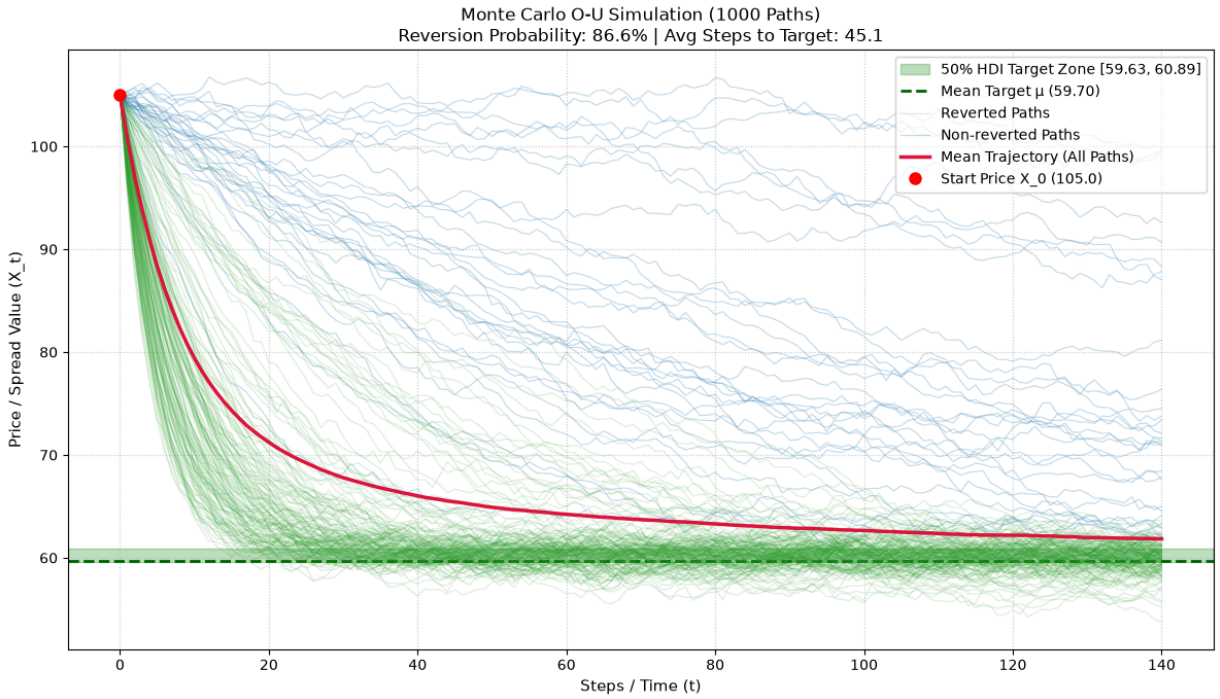
Buy Signal Execution and Position Sizing

Once a trade candidate passes the filtering stage, the system evaluates the mathematical expectancy of the trade and determines optimal capital allocation.

Path Simulation and Reversion Probability

To model the actual mechanics of the trade, the system simulates 2,000 discrete price paths using a vectorized Euler-Maruyama discretization of the Ornstein-Uhlenbeck (O-U) process over a 130-period horizon. By sampling parameter sets (μ, θ, σ) directly from the MCMC posterior, the simulation inherently accounts for parameter uncertainty. The system calculates the empirical probability (P_{revert}) that the asset will successfully enter the 50% Highest Density Interval (HDI) of the mean within the 130-tick lifespan.

Show code



Expected Value (EV) Calculation

Using the simulated reversion probability, the framework strictly limits market entries to trades possessing a positive Expected Value (EV). The theoretical EV per share is calculated as:

$$EV = (P_{revert} \times \text{Potential Gain}) - ((1 - P_{revert}) \times \text{Potential Loss})$$

Where:

- **Potential Gain** is the absolute distance between the current price and the target equilibrium price (the midpoint of the HDI zone).
- **Potential Loss** is defined as an adverse price excursion equal to $3.5 \times \bar{\sigma}$, representing a structural failure of the model.

If the calculated EV is negative, or if the current price already resides inside the equilibrium zone, the system aborts the trade, returning a recommended size of zero.

Confidence-Scaled Capital Allocation

For trades demonstrating a positive EV, capital is allocated using a dynamic risk budgeting approach. The system limits maximum exposure on any single trade to 2% of total portfolio capital. The base position size is calculated by dividing this risk budget by the potential loss per share.

To optimize capital efficiency, this base size is then linearly scaled by the model's confidence (P_{revert}). Consequently, lower-probability setups consume a smaller fraction of the risk budget, while highly probable reversions utilize the maximum allowable allocation. Finally, the system verifies that the calculated notional value does not exceed available cash.

Exit Mechanics and Stop-Loss Framework

The exit methodology within this framework relies on continuous evaluation of the posterior distributions to ensure positions are closed either when statistical mean reversion is achieved or when structural model assumptions are invalidated.

Take-Profit: The 50% HDI Target Zone

Rather than utilizing a static price target, the take-profit mechanism is dynamically governed by the Highest Density Interval (HDI) of the expected mean (μ). For each period, the model calculates the upper and lower bounds containing 50% of the μ posterior distribution's probability mass.

When the current price (x_t) crosses into this 50% HDI box, it triggers a take-profit signal. This approach allows the system to execute the exit with a 50% statistical confidence that the asset has successfully reverted to its true population mean.

Distribution Health and Error Bounding

Before a take-profit or stop-loss order is executed based on distribution metrics, the model must validate the tightness and health of the underlying posterior. This acts as a secondary filter: the system calculates the 85% credible interval (spanning from the 7.5th to the 92.5th percentile) of the distribution.

The exit logic is only authorized if the total width of this 85% mass is less than 3.0 units. By enforcing this strict threshold, the system guarantees that the posterior distribution is highly concentrated, mathematically capping the parameter estimation risk.

Stop-Loss Logic

To protect the portfolio from structural breaks and regime shifts, the algorithm employs a dual-layered stop-loss framework:

- **Time-Based Stop (130 Ticks):** A position is automatically liquidated if the holding period exceeds 130 observational ticks. This time-stop is critical for maintaining alignment with the original Expected Value (EV) and position sizing calculations, which were simulated over a maximum 140-step horizon. Limiting the time exposure ensures the initial MCMC parameters accurately reflect the current market environment and prevents macroeconomic regime drift.
- **Structural Blowout (Z-Score ≥ 4):** If the asset's standardized deviation (Z-score) expands beyond 4 (and the distribution width is stable), the position is immediately aborted. An extreme deviation of this magnitude falls well outside normal Gaussian expectations, signaling a fundamental structural break in the underlying asset's behavior. At this extreme, the asset has entered an exploratory market regime that the original O-U model parameters can no longer safely or accurately forecast.

Backtest Performance & Results

Over the one-year backtest period, the mean-reverting algorithm successfully captured an **11.28% annual profit**. The system demonstrated highly efficient capital allocation, characterized by an exceptional profit factor and strictly managed downside risk.

Key Performance Metrics

Return & Efficiency

- **Win Rate:** 58.2% (across 18 total trades)
- **Profit Factor:** 4.50
- **Expectancy:** +0.97% per trade
- **Sharpe Ratio:** 1.17

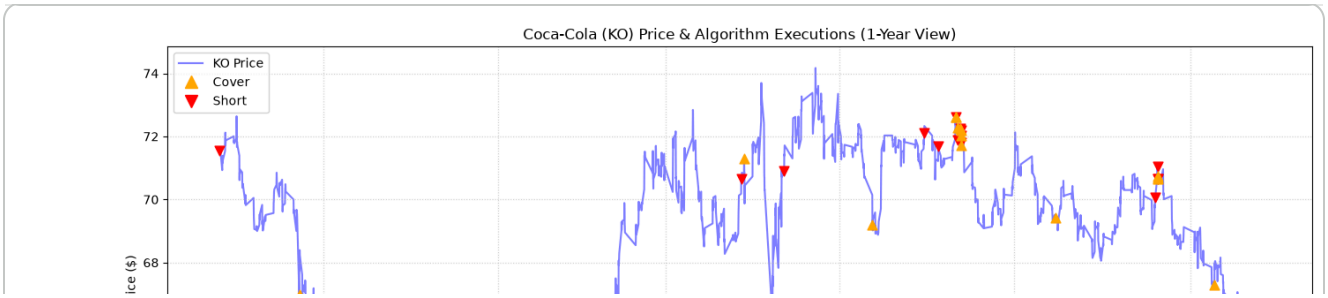
Risk Management

- **Average Win:** +2.33%
- **Average Loss:** -0.71%
- **Max Drawdown:** -1.30%
- **Average Time in Market:** 10 Days, 18 Hours

Trade Execution Analysis

As shown in the execution graph below, the algorithm consistently capitalized on localized downtrends by systematically selling high and buying low. Just as importantly, the risk-management logic successfully protected capital by preventing drawdowns when the Z-score flagged unpredictable volatility spikes.

Show code


Coca-Cola (KO) Price & Algorithm Executions (1-Year View)
KO Price
Cover
Short
74
72
70
68